SOFTOMATIC
TECHNOLOGY SERVICES

EXAMATE · SYSTEM DESIGN · ELITE + BATCH 1

Lesson 2 · Client–Server, HTTP and API Design

How a request really travels, how to read and write one, and how to design an API that survives a lost reply. Told through two stories: a payment tapped inside a train tunnel, and a restaurant with a very forgetful waiter.

MODULE
System Design · Part A
Foundations

TIME
About 80 minutes
plus 45 minutes of practice problems

YOU NEED
Lesson 1
No networking background needed.

YOUR ROUTE TODAY

01 Pay ₹500 in a tunnel
02 DNS, TCP, TLS, HTTP
03 Methods and status codes
04 Idempotency and pagination
05 **REST, gRPC, GraphQL and the API sketch**

LESSON 2 AT A GLANCE

What you will be able to do

Every system you design later talks over HTTP. This lesson gives you the vocabulary, the numbers and the habits that make an API **predictable, retry-safe and easy to explain**.

URL to page, with numbers
Explain what happens between typing a URL and seeing a page, and price each step.

Read and write HTTP
Read a request and a response; pick the right method and status code.

Idempotency
Explain it, and make a payment-style API safe to retry.

Offset vs cursor
Choose a pagination style and show exactly why one breaks.

REST, gRPC, GraphQL
Compare the three and pick one for a given system.

A 5-minute API sketch
Four or five endpoints for a prompt like "URL shortener", with errors, pagination and rate limits.

Subject / module	System Design / Part A: Foundations
Difficulty	Beginner
Prerequisites	Lesson 1 (what a design round is, requirements, the 5-step flow). No networking background needed.
Next lesson	Lesson 3: Relational databases for designers
Companies covered	Amazon, Microsoft, Google, Cisco, Adobe, Atlassian, Uber, Flipkart

How to read the evidence labels. Every *reported* interview question carries one label. Practice problems and quiz questions are **original** and unlabeled. **[S#]** means a page opened in full; **[X#]** means text seen only in search results. Both lists are on the Sources pages.

SOURCE-CONFIRMED
A dated first-person report naming the company and the question, Oct 2024 to Oct 2026, opened in full.

CANDIDATE-REPORTED
Compilations, aggregator pages, older than 24 months, undated, or text seen only in search extracts.

PATTERN-ONLY
Commonly asked, but no company source, so no company name is attached.

Pause and predict
Guess before you read on

What goes wrong
The mistake people actually make

Interview mode
Sentences you can say aloud

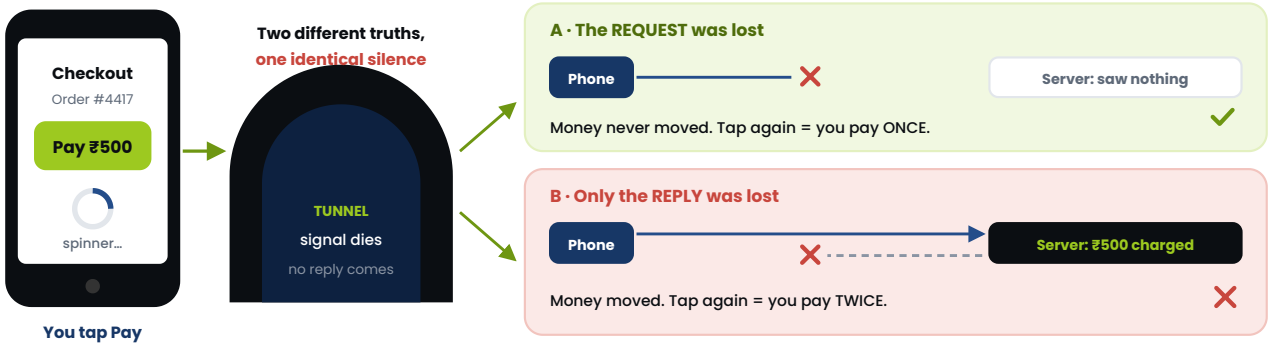
Tiny challenge
Thirty seconds, answer included

Memory hook
A line that sticks

A SPINNER IN A TUNNEL

You tap Pay ₹500. The signal dies.

The spinner turns, the train goes into a tunnel, the signal dies. You do not know whether the payment went through. You tap **Pay** again.



From the phone, A and B look exactly the same: a spinner, then a timeout. Safe APIs are designed around this one fact.

Same silence, two different truths. Either the first request never reached the server (you are about to pay once), or it reached the server, the money moved, and only the *reply* was lost (you are about to pay twice).

Your phone **cannot tell these two cases apart**. That one uncomfortable fact is why serious APIs have retry rules, idempotency keys and careful status codes. A Cisco interview report from July 2026 shows interviewers poking exactly this territory: while a senior candidate drew their own system, they were asked about rate limiting, circuit breakers, queues, sync vs async, and REST vs gRPC vs GraphQL [S6].

PAUSE AND PREDICT

If the app sends the *same* payment request twice, what would the server need to remember to charge only once?

Answer: Something that identifies "this exact payment attempt", plus the result it already gave. It needs a better memory than your cousin who "definitely already sent the UPI". We build that memory in Concept 6.

INTERVIEW MODE: SAY IT LIKE THIS

- A network can lose the request or the reply, and the client cannot tell which.
- Safe APIs are designed around that: idempotent operations, idempotency keys for non-idempotent ones, and clear status codes.

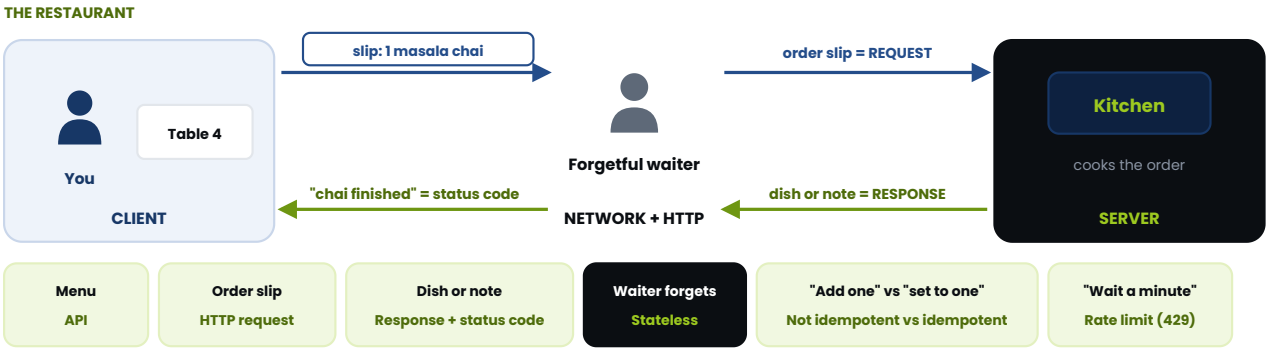
MEMORY HOOK: THE ONE SENTENCE

"Lost request or lost reply? The client can't know." Every retry rule in this lesson exists because of that sentence.

BEFORE THE JARGON

A restaurant with a very forgetful waiter

You (the **client**) sit at a table. The kitchen (the **server**) cooks. Between you sits a waiter (the **network and HTTP**). The **menu** is the API: it lists exactly what you may order and in what form.



Six mappings to keep. The rest of the lesson is these six ideas made precise.

The order slip (the request): "Table 4, one masala chai." It has a *verb* (add, change, cancel), a *thing* (chai) and details (less sugar).

The reply: a dish, or a note ("chai finished", "you cannot order that", "kitchen on fire, come back later"). Those notes are **status codes**.

Forgetful: the waiter remembers nothing between trips, so every slip carries everything: table number, full order, your name. That is **statelessness**.

"Add one chai" written twice gets two chais. **"Table 4 order is: one chai"** written twice still means one. That difference is **idempotency**.

Dropped slip vs dropped dish. If the waiter drops the slip, the kitchen never sees it. If the kitchen cooks but the waiter drops the *dish*, you never see it. Same silence from the table, different truth: the lost-reply problem.

A very long menu can be shown **a page at a time** (pagination), and a kitchen can say "too many orders from table 4, wait a minute" (**rate limiting**).

WHAT GOES WRONG

Where the analogy breaks. A real waiter can *see* you and re-ask; a network message cannot. And real kitchens do not duplicate themselves; servers do (many copies behind one address), which is why "which kitchen cooked my order?" becomes a real question later.

INTERVIEW MODE: SAY IT LIKE THIS

- An API is a contract listing the requests a system accepts.
- HTTP is stateless: each request carries all it needs.
- Operations differ in whether repeating them is safe, and that decides how clients may retry.

TINY CHALLENGE

Pick an app you use. Write one request that is safe to repeat and one that is dangerous to repeat. Answer: "Show my order history" is safe; "place this order" is dangerous unless the server can spot the repeat.

EVERY WORD, ONE LINE

Vocabulary primer

Each term is defined here before it is used. *Cache, load balancer, queue* and *database* are only named here; each has its own lesson.

Client / server
The client asks; the server answers. Carried over from Lesson 1.

Protocol
An agreed set of rules for how two programs talk.

HTTP
HyperText Transfer Protocol: the request-and-reply rules the web and most APIs use.

HTTPS
HTTP carried inside an encrypted connection (using TLS).

URL
The address of a thing, such as https://api.example.com/v1/links/ab12.

Resource
A "thing" an API exposes, such as a link, a user or an order.

Method (verb)
What to do with the resource: GET, POST, PUT, PATCH, DELETE.

Header
A named extra detail on a request or reply, such as Content-Type.

Body
The main data in a request or reply, often JSON.

JSON
JavaScript Object Notation: a text format for data, like {"name":"Asha"}.

Status code
A 3-digit number in a reply that says what happened (200, 404, 429, 500…).

Query parameter
A key-value pair after ? in a URL, like ?limit=20.

Endpoint
One method plus one URL pattern, such as POST /v1/links.

API
Application programming interface: the list of endpoints and their rules.

REST
Representational State Transfer: name resources with URLs, act on them with standard HTTP methods.

Stateless
The server keeps no memory of earlier requests; each request is complete by itself.

Safe method
Only reads; changes nothing on the server (GET is safe).

Idempotent
Doing it many times leaves the server in the same state as doing it once.

Idempotency key
A unique ID the client attaches so the server can spot a repeat of the same request.

Pagination
Returning a long list in small pages instead of all at once.

Offset pagination
"Skip the first N items, give me the next L."

Cursor pagination
"Give me L items after this specific item."

RPC
Remote procedure call: calling a function on another machine as if it were local.

gRPC
A framework for RPC; by default it uses Protocol Buffers.

Protocol Buffers
protobuf: a compact binary format with a schema file, gRPC's default.

GraphQL
A query language for APIs where the client lists exactly which fields it wants.

Over- / under-fetching
Getting more data than needed / too little, forcing extra calls.

DNS
Domain Name System: the phone book that turns example.com into an IP address.

IP address
The numeric address of a machine on a network.

TCP
Transmission Control Protocol: a reliable, ordered connection between two machines.

TLS
Transport Layer Security: encrypts the connection so others cannot read or alter it.

RTT
Round-trip time: how long a message takes to go to the server and back, in ms.

Timeout
The longest a client waits before giving up.

Retry / backoff
Sending again after a failure / waiting longer between each attempt.

Authentication (authN)
Proving *who you are*.

Authorization (authZ)
Deciding *what you are allowed to do*.

Rate limit / 429
A cap on requests per client; beyond it the server replies 429 "Too Many Requests".

INTERVIEW MODE: SAY IT LIKE THIS

- Use the exact words: "idempotent", "stateless", "cursor pagination", "status code", "idempotency key". Define each in one line if asked.

CONCEPT 1 · CLIENT, SERVER AND PRESSING ENTER

What happens when you type a URL

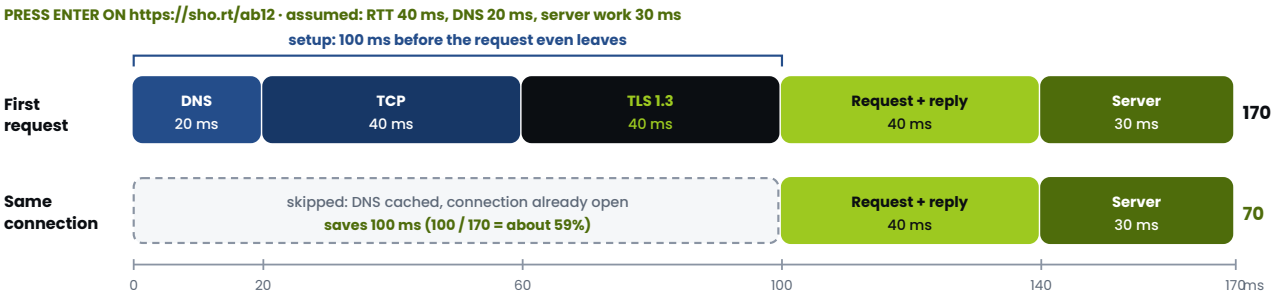
Every latency and failure discussion in later lessons points back to this picture. Our running example is a **short-link service**, the URL shortener reported in an Amazon round in June 2025 [S8].

1. **DNS lookup:** ask DNS for the IP address of sho.rt.
2. **TCP connection:** open a reliable connection to that IP address (one round trip).
3. **TLS handshake:** agree on encryption (about one round trip with TLS 1.3, about two with TLS 1.2; verify the version your server uses).
4. **HTTP request:** send it; the server processes it and replies (one more round trip plus server time).
5. **Render:** the browser uses the reply.

MEMORY HOOK: D-T-T-H: "DO TAKE THE HIGHWAY"

DNS finds the address, **T**CP opens the road, **T**LS locks the doors, **H**TTP finally carries the parcel. Only the last step does useful work.

An Amazon candidate says they were asked "What happens when you type a URL into your browser?" and lists the same steps [X3]. That is a LinkedIn post from about 2023, so it is **CANDIDATE-REPORTED** and older than 24 months.



Three round trips (TCP, TLS, request) happen before any useful data arrives. That is why clients keep connections open.

All numbers are assumptions: RTT 40 ms, DNS 20 ms (not cached), TLS 1.3, server work 30 ms.

STEP	TIME	STEP	TIME
DNS	20 ms	Request and reply	40 ms
TCP connect	40 ms	Server work	30 ms
TLS handshake	40 ms	**First request total**	**170 ms**
		Next request, same open connection (40 + 30)	**70 ms**

CONCEPT 1 · CONTINUED

Reuse saves 100 ms. Silence tells you nothing.

BACK IN THE TUNNEL

The reused connection saves **100 ms, about 59%** (100 / 170). But speed is not the scary part. When the train enters the tunnel, the phone waits until its **timeout** and gives up, and a timeout tells you nothing about whether the server did the work. That is the hook's problem, and it starts here.

WHAT GOES WRONG

DNS fails: the browser says it cannot find the site. **The server is slow or down:** the client waits until its timeout and then errors. **A timeout tells you nothing** about whether the server acted. Silence from a server is like silence from your crush: it does not mean yes or no.

PAUSE AND PREDICT

With the same numbers, which single change helps more: a server that is 10 ms faster, or a client that keeps its connection open?

Answer: Keeping the connection open: it removes DNS, TCP and TLS, 100 ms, versus 10 ms of server time.

INTERVIEW MODE: SAY IT LIKE THIS

- "DNS gives the IP, TCP opens a connection, TLS encrypts it, then HTTP carries the request and reply."
- "Opening a connection costs round trips, so clients reuse connections."
- "A timeout means I do not know whether the server acted."

TINY CHALLENGE

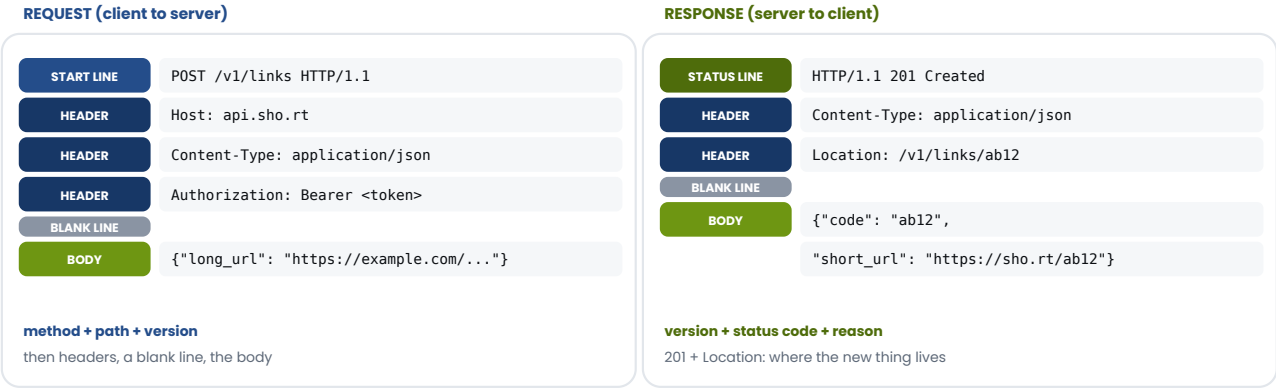
With RTT = 100 ms and the same other numbers, what is the total of a first request? **Answer: DNS 20 + TCP 100 + TLS 100 + request 100 + server 30 = 350 ms.**

LAYER	WHAT IT GIVES YOU	COST IN OUR EXAMPLE
DNS	The IP address for a name	20 ms (0 when cached)
TCP	A reliable, ordered connection	1 round trip = 40 ms
TLS	Encryption: nobody can read or alter it	1 round trip with TLS 1.3 = 40 ms
HTTP	The request and the reply	1 round trip + server work = 70 ms

CONCEPT 2 · WHAT AN HTTP MESSAGE LOOKS LIKE

If you can't read it, you can't design or debug it

A request has a **method + path + version**, then **headers**, then an optional **body**. A reply has a **status line** (version + status code + reason), headers, and an optional body.



The order slip and the waiter's note. A blank line always separates the headers from the body.

```
POST /v1/links HTTP/1.1
Host: api.sho.rt
Content-Type: application/json
Authorization: Bearer <token>

{"long_url": "https://example.com/
 a/very/long/path"}
```

```
HTTP/1.1 201 Created
Content-Type: application/json
Location: /v1/links/ab12

{"code": "ab12",
 "short_url": "https://sho.rt/ab12"}
```

The headers named here (Content-Type, Authorization, Location, Retry-After) are all listed in MDN's header reference [S1 nav list]. (The JSON body is shown on two lines only to fit the page.)

Worked example (size). The request body is about 60 bytes of JSON; headers typically add a few hundred bytes. So tiny API calls are dominated by **round-trip time, not size**: a good reason to avoid many tiny sequential calls (Concept 8).

WHAT GOES WRONG

Wrong Content-Type (JSON labelled as form data), or a missing Authorization header: the server replies with a **4xx** code (client error), not a crash. A request is basically a letter with extra steps.

TINY CHALLENGE

Write the request line and one header for "fetch link ab12". Answer: `GET /v1/links/ab12 HTTP/1.1` with `Host: api.sho.rt` (or an Authorization header).

INTERVIEW MODE: SAY IT LIKE THIS

- "A request is method, path, headers, body. A reply is status code, headers, body."
- "Content-Type says what the body is; Authorization carries credentials."

CONCEPT 3 · RESOURCES, URLS AND REST

Nouns in the path, verbs in the method

Without a naming habit, every team invents URLs like /doTheThing and nobody can guess the API.

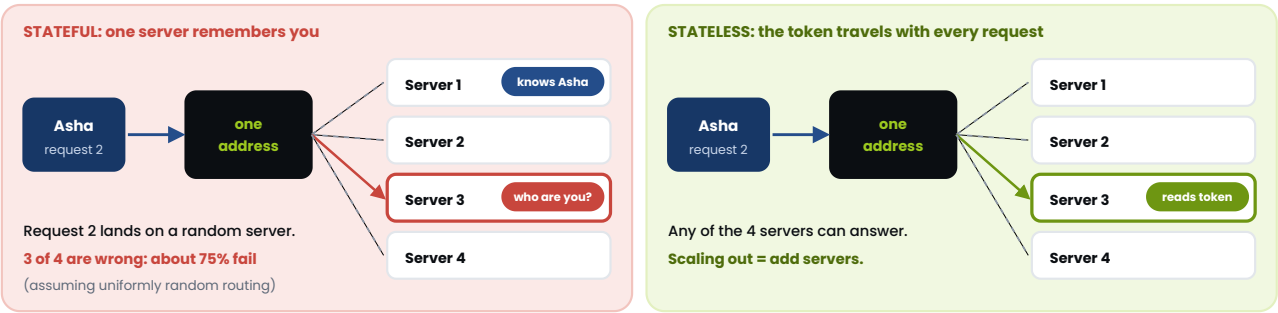
REST style: name **resources** with nouns in URLs, act on them with standard methods, keep the server **stateless**, and exchange representations (usually JSON).

DataCamp's 2026 interview guide notes that most APIs called "REST" are really REST-like, at **level 2 of the Richardson Maturity Model**: they use resources and HTTP verbs but not full hypermedia [X6].

GOAL	ENDPOINT
Create a link	`POST /v1/links`
Read one link	`GET /v1/links/ab12`
List my links	`GET /v1/links?limit=50`
Delete a link	`DELETE /v1/links/ab12`

Not /createLink or /getLinks.

WHY HTTP IS STATELESS: 4 IDENTICAL SERVERS BEHIND ONE ADDRESS



Worked example. If a server kept "who this user is" in its own memory, request 2 might hit a server that never met you. With 4 servers, a stateful design fails the second request about 75% of the time (3 of 4 are wrong), assuming uniformly random routing.

WHAT GOES WRONG

Verbs in URLs, inconsistent plural/singular, or hiding actions inside POST with unclear meaning. Interviewers mostly care that you are consistent and can justify the choice. Verbs in URLs are like naming every Swiggy order "IWantFoodNow".

TINY CHALLENGE

Write the four endpoints for a notes resource.

Answer: POST /v1/notes, GET /v1/notes/{id}, GET /v1/notes?limit=50, DELETE /v1/notes/{id}.

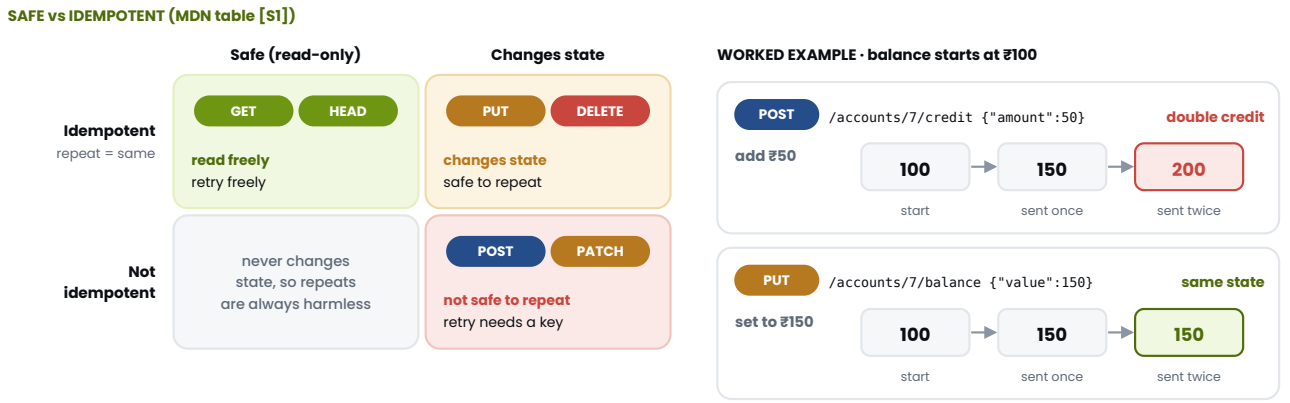
INTERVIEW MODE: SAY IT LIKE THIS

- "REST: nouns as URLs, standard methods, stateless, JSON representations."
- "Stateless means any server can handle any request, which makes horizontal scaling simple."

CONCEPT 4 · METHODS: SAFE, IDEMPOTENT, WHICH ONE

The method tells everyone whether repeating is harmless

Browsers, proxies and retry logic all read the method to decide whether a request may be sent again.



Idempotent is not the same as safe: DELETE changes state (not safe) but deleting twice still leaves it gone (idempotent).

One grid, six methods. Green: retry freely. Amber: changes state but repeats safely. Red: a blind retry can do the work twice.

METHOD	SAFE	IDEMPOTENT	TYPICAL USE
GET	Yes	Yes	Read a resource
HEAD	Yes	Yes	Like GET without the body
PUT	No	Yes	Replace a resource entirely
DELETE	No	Yes	Remove a resource
POST	No	No	Create, or trigger an action
PATCH	No	No	Change part of a resource

MDN's table [S1]. MDN describes PUT as replacing all current representations of the target with the request content, and POST as submitting an entity, often causing a state change or side effects [S1].

MEMORY HOOK: "SET TO" VS "ADD"

PUT is the friend who says **"set it to 150"**. POST is the friend who says **"add 50, bro"**. Only one of them is safe to nag twice.

WHAT GOES WRONG

Calling a non-idempotent POST "safe to retry" because "it usually works". And **idempotent does not mean safe**: DELETE is idempotent (deleting twice leaves the item gone) but not safe (it changes state) [S1].

TINY CHALLENGE

Is DELETE /links/ab12 safe? Idempotent? Answer: Not safe; idempotent.

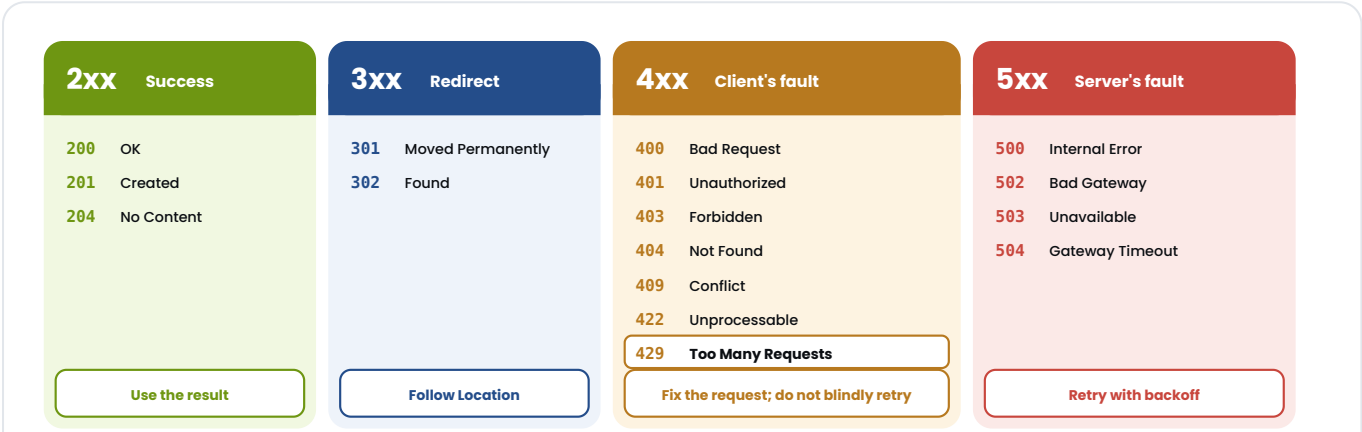
INTERVIEW MODE: SAY IT LIKE THIS

- "GET is safe and idempotent. PUT and DELETE are idempotent but not safe. POST and PATCH are neither."
- "PUT replaces the whole resource; PATCH changes part of it; POST creates or triggers."

CONCEPT 5 · STATUS CODES

The waiter's note drives the client's next move

Show an error, retry, log in again, or give up? The status code decides. Codes come in classes: **2xx** success, **3xx** redirect, **4xx** the client did something wrong, **5xx** the server failed (the class meanings are standard usage; MDN calls 429 a client-error code [X2]).



429 is a 4xx code, but the fix is to wait (Retry-After), not to change the request. 401 = who are you? 403 = I know you, but no.

Four families, four next moves. Code names are from MDN's status list [S1 nav list].

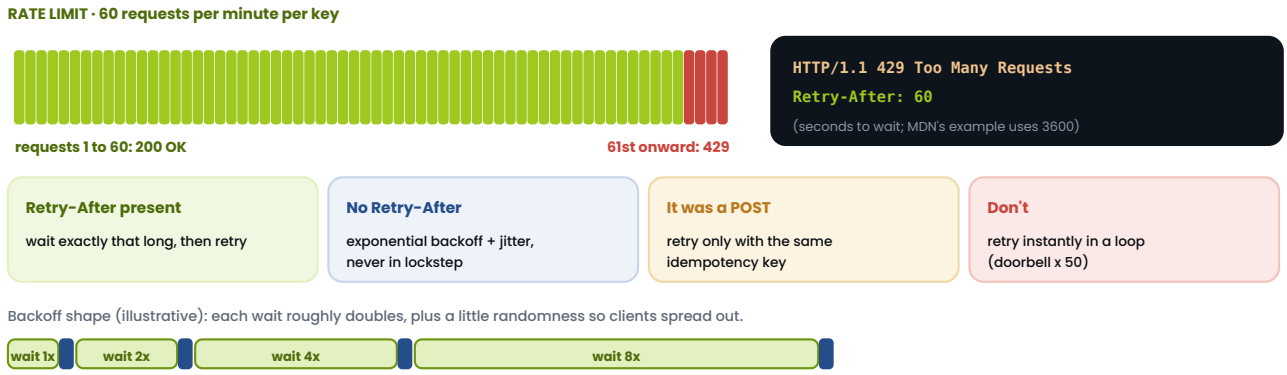
CODE	NAME (MDN LIST)	USE IT WHEN
200	OK	Request worked and you return data
201	Created	A new resource now exists (add a Location header)
204	No Content	Worked, nothing to return (often DELETE)
301 / 302	Moved Permanently / Found	Redirect: permanent vs temporary
400	Bad Request	The request is malformed
401	Unauthorized	Credentials missing or invalid (in practice: "not authenticated")
403	Forbidden	You are known but not allowed (in practice: "not authorized")
404	Not Found	No such resource
409	Conflict	The request clashes with current state
422	Unprocessable Content	Well-formed but fails validation
429	Too Many Requests	You are over the rate limit
500	Internal Server Error	The server hit a bug
502 / 503 / 504	Bad Gateway / Service Unavailable / Gateway Timeout	A proxy or upstream failed / overloaded / too slow

CONCEPT 5 · CONTINUED

Redirects, honest codes and 429

The "use it when" column on the previous page is standard practice; MDN's per-code pages carry the precise definitions, and they were not each opened for this lesson.

429 can carry a Retry-After header saying how long to wait; MDN documents Retry-After for 429, 503 and redirect replies [X2]. The 429 code itself is specified in RFC 6585 section 4, as the pt-BR MDN page shows [X2].



The kitchen says "wait a minute". The 60-per-minute limit is the rate limiter from Lesson 1.

Worked example (the short-link redirect). GET /ab12 should answer with a redirect. A **301** (permanent) can be cached by browsers, so later clicks may never reach your server and your click counts miss them; a **302** (temporary) usually keeps traffic flowing through you. (Browser caching of 301 is typical behaviour; verify against your target browsers.)

WHAT GOES WRONG

Returning **200 with {"error": ...}** in the body: monitoring, retries and caches all think it worked. Or returning **500 for bad user input**: your alerts fire for the user's typo. Returning 200 with an error inside is a shopkeeper saying "sure!" while shaking his head.

MEMORY HOOK: THE FOUR FAMILIES

2 = "yes, here you go". **3** = "go over there". **4** = "you messed up". **5** = "we messed up". And **401 = "Who?"**, **403 = "No."**

TINY CHALLENGE

Which code for "the short code does not exist"? For "you sent invalid JSON"? Answer: 404. 400.

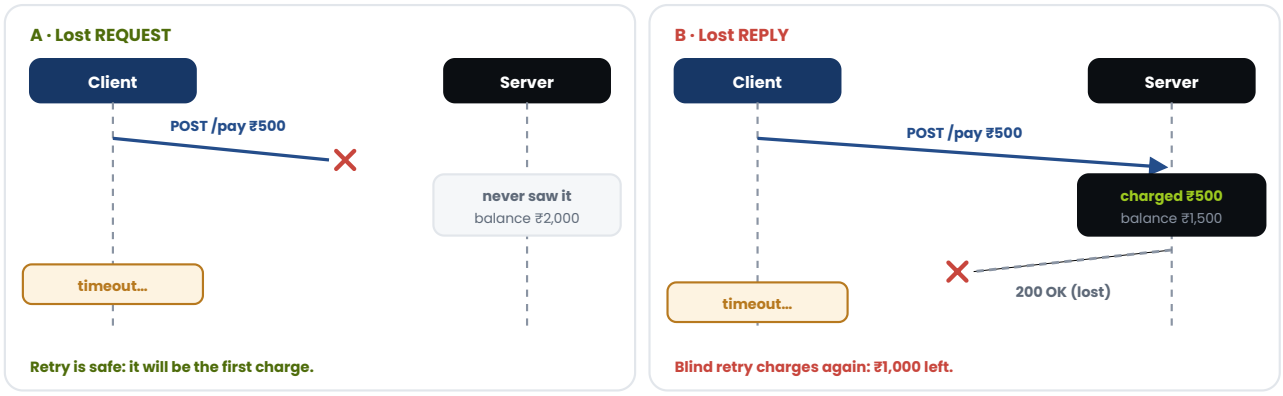
INTERVIEW MODE: SAY IT LIKE THIS

- "2xx success, 3xx redirect, 4xx client error, 5xx server error."
- "401 means not authenticated, 403 means authenticated but not permitted."
- "429 with Retry-After tells the client when to retry."

CONCEPT 6 · IDEMPOTENCY AND SAFE RETRIES

Back to the tunnel: the server needs a memory

The client cannot tell whether its request or only the reply was lost. Retrying a non-idempotent request could do the work twice.



The client's view is identical in both panels: request sent, then a timeout. Only an idempotency key lets the server tell them apart.

Balance starts at ₹2,000. Left: nothing happened, a retry is the first charge. Right: the charge happened, a blind retry is the second.

How it works. Attach a unique **idempotency key** to a POST. The server remembers the key and the result:

1. **Key never seen:** do the work, store the key and result, reply.
2. **Key already seen:** do **not** repeat the work; reply with the stored result.

MEMORY HOOK: THE BANK TOKEN

An idempotency key is a **bank token number**: show the same token and they find your file, they do not open a new one.

MDN's description matches: the client can resend the same request with the same key, and the server knows to perform the action only once [S2].

WHAT REAL SYSTEMS DO

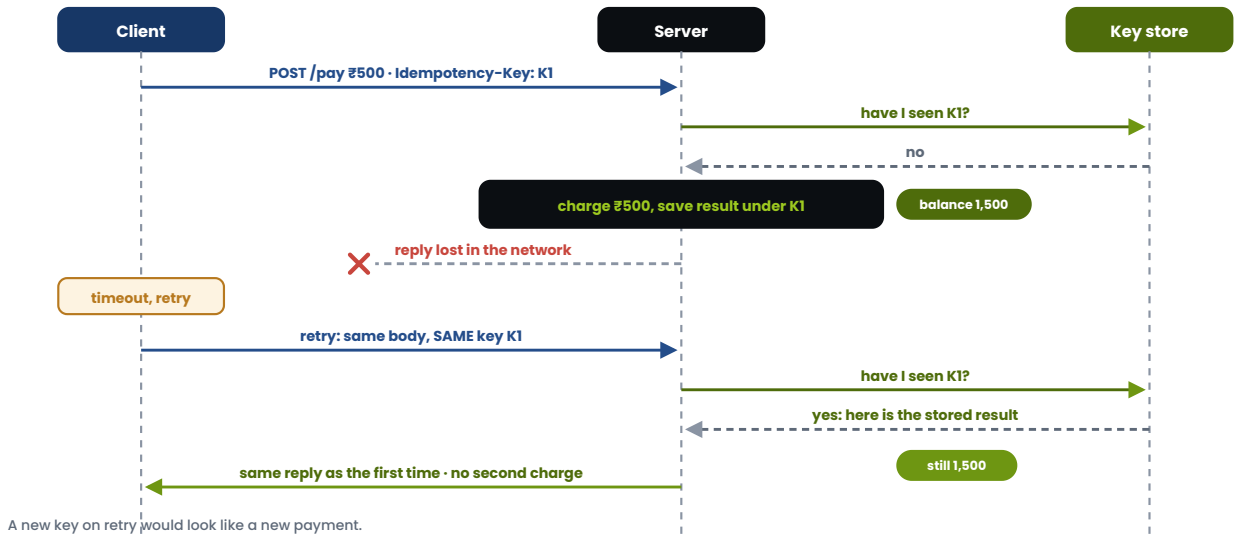
MDN records that the Idempotency-Key header is **non-standard and experimental**, with a draft specification, so do not call it "part of HTTP" [S2]. Stripe documents a real implementation: it saves the status code and body of the *first* request for a key, **whether it succeeded or failed**, and returns the same result for later requests with that key; keys are **up to 255 characters**; keys may be removed once **at least 24 hours** old; and a reused key with **different parameters causes an error** [S3].

TINY CHALLENGE

Why can't the server use the *request body* alone as the key? Answer: Two genuine identical purchases would be merged.

CONCEPT 6 · CONTINUED

The idempotent retry, step by step



The retry reuses the same key K1. A new key would look like a new payment.

TIME	EVENT	BALANCE
t0	Client sends pay ₹500 with key K1; server charges	1,500
t1	Reply lost; client times out	1,500
t2	Client retries with K1; server finds K1, replies with the stored result	**1,500** (not 1,000)
t3	Without a key the retry would charge again	would be 1,000

SERVER ERROR CASE (MDN [S2])	CODE
A required key is missing	400
A request with the same key is still being processed (the client waits and retries)	409
The key is reused with a different payload (when the server fingerprints requests)	422

WHAT GOES WRONG

(a) The client generates a *new* key on every retry, which defeats the point. **(b)** Two requests with the same key arrive at the same moment; without a conflict rule both might execute. **(c)** Keys expire, so a very late retry is treated as new. **(d)** Retrying on 429 or timeout without a key: an ApyHub explainer warns that a 429 can arrive after the work was already done, so retry POST only with a key or a way to confirm the original did not apply [X7].

INTERVIEW MODE: SAY IT LIKE THIS

- "GET, PUT and DELETE are idempotent by definition; POST needs an idempotency key."
- "The server stores key plus result; a repeat returns the stored result without redoing the work."
- "The client must reuse the same key on retry. Keys expire, and concurrent same-key requests need a conflict rule."

CONCEPT 7 · PAGINATION: OFFSET VS CURSOR

Reading a newspaper while someone inserts pages at the front

A list of 10 million rows cannot be returned in one reply. Pagination returns it in pages, but the way you page decides whether users see duplicates or misses.

Offset

`GET /posts?offset=10&limit=10` means skip 10, return 10. It counts **positions**.

Cursor

`GET /posts?limit=10&starting_after=post_91` means return 10 items after the item with that ID. Stripe's list APIs work this way: limit, starting_after, ending_before, a has_more flag, and limit between 1 and 100 [S4].



The offset moves with inserts, but the cursor points at a fixed item. Page 1 showed ids 100 to 91; then 103, 102, 101 arrive at the top. Position 11 is now id 93.

Worked example. Page size 10, three posts inserted between page 1 and page 2. Offset paging repeats **3 items** (ids 93, 92, 91), so the user sees **7 new items instead of 10**. If 3 items were *deleted* from page 1's range, offset paging would *skip* 3 unseen items. Cursor paging avoids both, because it continues from a named item.

Page count. 1,000 links with limit=50 need 1,000 / 50 = **20 pages**.

Cost. Offset paging at offset=1,000,000 asks the database to get past a million rows before returning 10; cursor paging can jump straight to the cursor's position if it is indexed. (Typical database behaviour; confirm with EXPLAIN on your database.)

MEMORY HOOK: SEATS VS BOOKMARKS

Offset counts seats ("the 11th seat"); if people squeeze in, the 11th seat holds someone else. **A cursor is a bookmark** in the page itself.

CONCEPT 7 · CONTINUED

The cursor must sort on a unique key

WHAT GOES WRONG

Offset on a fast-changing feed gives duplicates and gaps. **A cursor built on a column that is not unique**: two posts with the same timestamp can be skipped; use a unique tiebreaker such as the ID, sorting by (created_at, id).

Uber candidates reported being asked how to paginate a REST API, though that post's style looks templated, so treat it with care [X1].

```
# Stripe-style cursor list: the reply says whether more pages exist [S4]
GET /v1/links?limit=50&starting_after=ab12

{"data": [ ...50 links... ], "has_more": true}
```

PAUSE AND PREDICT

Your feed sorts by created_at only. Two posts were created in the same millisecond and the page boundary falls between them. What can happen?

Answer: The cursor "after created_at = T" skips the second post (or repeats it, depending on the comparison). Sort by (created_at, id) and make the cursor carry both.

INTERVIEW MODE: SAY IT LIKE THIS

- "Offset breaks when data changes between pages and gets slower for deep pages."
- "Cursor pagination continues after a specific item, so it is stable and fast; the response says whether more pages exist."
- "The cursor must sort on a unique key."

	OFFSET	CURSOR
Points at	A position ("skip N")	A named item ("after id 91")
New items arrive between pages	Duplicates (3 inserts = 3 repeats)	No duplicates
Items deleted between pages	Skips unseen items	Nothing skipped
Deep pages	Database gets past N rows first	Jumps to the cursor if indexed
Needs	Nothing special	A unique sort key; reply says has_more

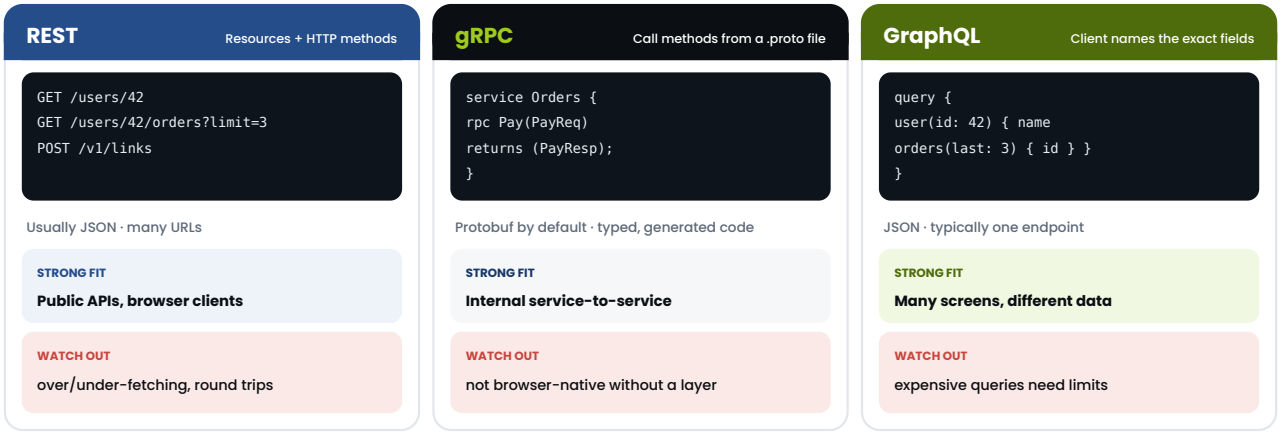
TINY CHALLENGE

5 items are deleted from page 1's range before you request offset page 2 (limit 10). How many unseen items do you skip? Answer: 5.

CONCEPT 8 · REST VS GRPC VS GRAPHQL

"Which API style?" is a standard follow-up

Cisco asked it explicitly: REST vs gRPC vs GraphQL came up while a senior candidate whiteboarded a system [S6].



Three models of the same idea. Snippets are illustrative.

	REST	GRPC	GRAPHQL
Model	Resources and HTTP methods	Call methods defined in a .proto file as if local	Client sends a query naming exact fields
Default data format	Usually JSON	Protocol Buffers by default (JSON possible) [S5]	JSON responses (typical)
Contract	Informal or OpenAPI	.proto schema with generated client and server code [S5]	Typed schema
Endpoints	Many URLs	Many methods on a service	Typically one endpoint [X5]
Strong fit	Public APIs, browser clients	Service-to-service calls inside a company	Varied client screens needing different data
Watch out	Over/under-fetching, many round trips	Not browser-native without extra layers	Expensive queries need limits (depth, complexity) [X5]

WHAT GOES WRONG

Choosing GraphQL and forgetting that a client can send a very expensive nested query; choosing gRPC for a browser-facing API without planning the extra layer; or claiming "gRPC is always faster so always use it". Picking GraphQL because it is trending is how a side project ends up with a 400-line schema for three buttons.

MEMORY HOOK: WHO IS CALLING?

REST for strangers (public, browsers), **gRPC for family** (your own services, shared contract), **GraphQL for picky eaters** (each screen wants a different plate).

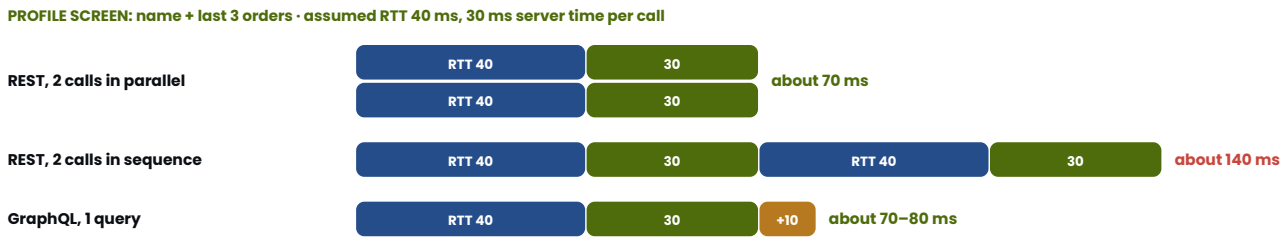
TINY CHALLENGE

An internal order service calls a payment service 5,000 times a second. Which style, and why?
Answer: gRPC: typed contract, compact messages, internal callers.

CONCEPT 8 · CONTINUED

Price it before you pick it

The gRPC introduction says a client can call a method on a server on a different machine "as if it were a local object", and that services are defined in ordinary proto files [S5]. Secondary explainers say gRPC runs over HTTP/2, giving multiplexing and streaming [X4]; the official intro page does not say so, so treat it as secondary.

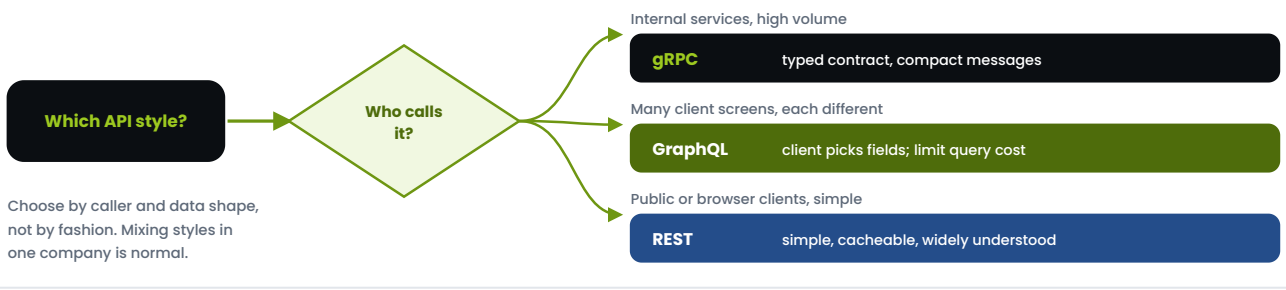


Worked example: a mobile profile screen needs the user's name and their last 3 orders. The +10 ms for GraphQL is an assumed bit of extra server work.

APPROACH	CALLS	TIME
REST, two calls **in parallel** (GET /users/42, GET /users/42/orders?limit=3)	2, parallel	about 70 ms (40 + 30)
REST, two calls **in sequence** (second depends on first)	2, sequential	about 140 ms (70 + 70)
GraphQL, one query asking for both	1	about 70–80 ms

THE HONEST LESSON

GraphQL saves round trips mainly when REST calls would be **dependent**; if you can fire the REST calls in parallel, the gap shrinks. Say that in an interview and you sound senior.



Choose by caller and data shape, not by fashion. Mixing styles in one company is normal.

INTERVIEW MODE: SAY IT LIKE THIS

- "REST for public simple APIs, gRPC for internal service-to-service with a typed contract, GraphQL when clients need flexible field selection."
- "I would state the trade-off: REST may over- or under-fetch, GraphQL needs query-cost limits, gRPC needs tooling for browsers."

CONCEPT 9 · AUTHENTICATION, AUTHORIZATION, HTTPS

Who are you, and may you?

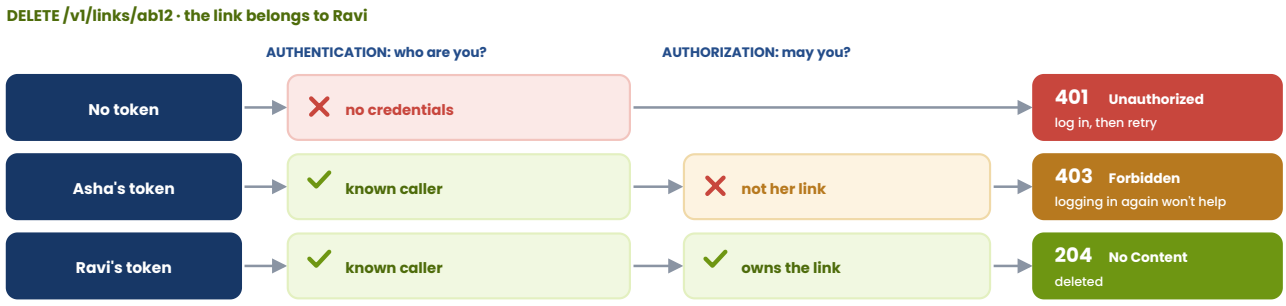
An API must know who is calling and what that caller may do. This is a likely gap interviewers sometimes reach, so here is the 10-minute version.

Authentication asks "who are you?". Typical answer: the client sends a credential in the Authorization header (for example `Authorization: Bearer <token>`); MDN lists Authorization among the standard request headers [S1 nav list].

Authorization asks "are you allowed to do this?". A user can be authenticated and still not allowed to delete someone else's link: that is the 403 case.

HTTPS encrypts the connection with TLS so others cannot read or alter the request in transit.

Session vs token (two common designs). In a *session*, the server stores who you are and gives you an ID (often in a cookie); in a *token* design, the client holds a signed token the server can verify without a lookup. A session needs shared storage across servers; a token is hard to revoke before it expires. No source was opened for this comparison; treat it as standard background and verify details for any specific framework.



Worked example (401 vs 403). No token: 401. Asha's valid token, but the link belongs to Ravi: 403. Ravi's token: 204.

WHAT GOES WRONG

Returning 404 for everything to "hide" resources is sometimes deliberate but confuses debugging; returning 200 on auth failure breaks clients; putting secrets in URLs, which get logged. A secret in a URL is like writing your ATM PIN on the receipt.

TINY CHALLENGE

A logged-in user opens another user's private page. Which code? Answer: 403.

INTERVIEW MODE: SAY IT LIKE THIS

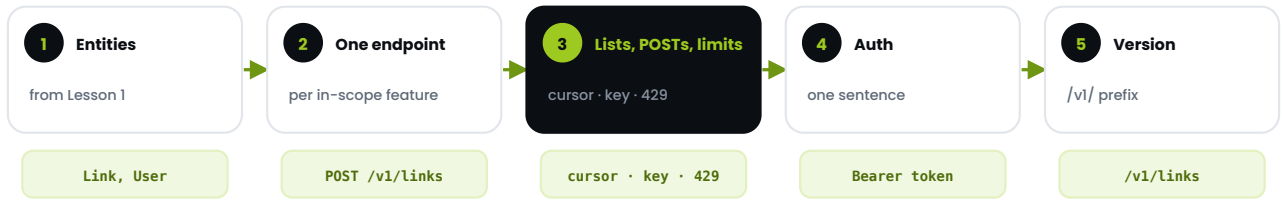
- "Authentication is who you are; authorization is what you may do."
- "401 if not authenticated; 403 if authenticated but forbidden."
- "Everything over HTTPS; credentials in headers, never in URLs."

CONCEPT 10 · DESIGNING AN API IN THE INTERVIEW

The 5-minute recipe

In Lesson 1's flow, step 3 is "data and API". You need a repeatable way to do it quickly.

DesignGurus advises sketching the four or five endpoints that matter, committing to one style, and not designing fifteen endpoints [S11]. Fifteen endpoints in a 45-minute round is a flex nobody asked for. The Atlassian SDE-2 report says to cover requirements, then API design including protocols, then database design, then list the services [S7].



Green row: the same step applied to the short-link service. Each endpoint = method + URL + request body + response + status codes.

Five steps. Step 3 is where most candidates lose marks: they forget pagination, retries and limits.

1. **List the entities** (Lesson 1).
2. **For each in-scope feature, write one endpoint:** method, URL, request body, response, status codes.
3. **Add:** pagination for every list, an idempotency story for every POST that moves money or creates something important, and a rate-limit reply (429 + Retry-After).
4. **Name authentication** in one sentence.
5. **Mention versioning** (a /v1/ path prefix is a common choice).

MEMORY HOOK: "EVERY ENGINEER LOVES A VERSION"

Entities, **E**ndpoints, **L**ists-keys-limits, **A**uth, **V**ersion.

INTERVIEW MODE: SAY IT LIKE THIS

- "I will define four or five endpoints with method, URL, request, response and error codes."
- "Every list gets cursor pagination; every important POST gets an idempotency key; the API returns 429 with Retry-After when limited."

TINY CHALLENGE

For "notes app", write the five endpoints. Answer: See Practice problem 5 for one full answer.

ONE WHITEBOARD, EVERY IDEA

The short-link API

Assumptions carried from Lesson 1's estimation habit: **100 million new links per month**; reads are **100 times** writes; a 30-day month has **2,592,000 seconds**.

LINE	CALCULATION	RESULT
Average create rate	100,000,000 / 2,592,000	about 39 per second
Average redirect rate	39 x 100	about 3,900 per second



The whole API on one board. The green-bordered row is the hot path.

ENDPOINT	REQUEST	SUCCESS REPLY	ERRORS
`POST /v1/links` + Idempotency-Key	{"long_url": "..."}	201 Created, Location: /v1/links/ab12, body with code	400 bad URL, 401, 422 key reused with different body, 429 + Retry-After
`GET /{code}`	none	302 Found + Location: <long URL>	404 unknown code, 410 Gone if expired
`GET /v1/links?limit=50&starting_after=<code>`	none	200 with data[] and has_more	401
`GET /v1/links/{code}`	none	200 link details	401, 403, 404
`DELETE /v1/links/{code}`	none	204 No Content	401, 403, 404

WHY EACH CHOICE

The narration that earns the marks

1. **POST with an idempotency key:** a retry after a lost reply returns the *same* code instead of making a second link, using the Concept 6 mechanism.
2. **302 for the redirect:** temporary redirects usually keep traffic flowing through the service so clicks can be counted; a 301 can be cached by browsers (Concept 5).
3. **Cursor list:** a user's link list changes as links are added, so offset would duplicate and skip; 1,000 links at limit=50 is 20 pages.
4. **429 with Retry-After:** the rate limiter from Lesson 1, 60 requests per minute per key, rejects extras with this reply.
5. **/v1/:** leaves room to change the API later without breaking old clients.

~39 /s	~3,900 /s	78 and 7,800 /s
creates, average	redirects, average	at the 2x peak assumed in Lesson 1's practice problem

The read-heavy ratio is why the redirect path is the one to optimise (Lesson 6).

SAY IT IN ONE BREATH

"Five endpoints: create with an idempotency key, redirect with 302, list with cursor pagination, get, delete. Errors use 4xx for client mistakes, 429 with Retry-After when limited. Reads dominate by 100 to 1, so I will optimise the redirect path."

```
# attempt 1 (reply lost) and the retry: SAME key
POST /v1/links HTTP/1.1
Idempotency-Key: K1
Content-Type: application/json

{"long_url": "https://example.com/..."}
```

```
# the retry gets the stored first result
HTTP/1.1 201 Created
Location: /v1/links/ab12

{"code": "ab12",
 "short_url": "https://sho.rt/ab12"}
```

PAUSE AND PREDICT

The interviewer adds: "Links expire after a year." What changes in your API?

Answer: GET /{code} returns 410 Gone for an expired link (or 404 if you treat unknown and expired alike), and the create body could accept an expiry time.

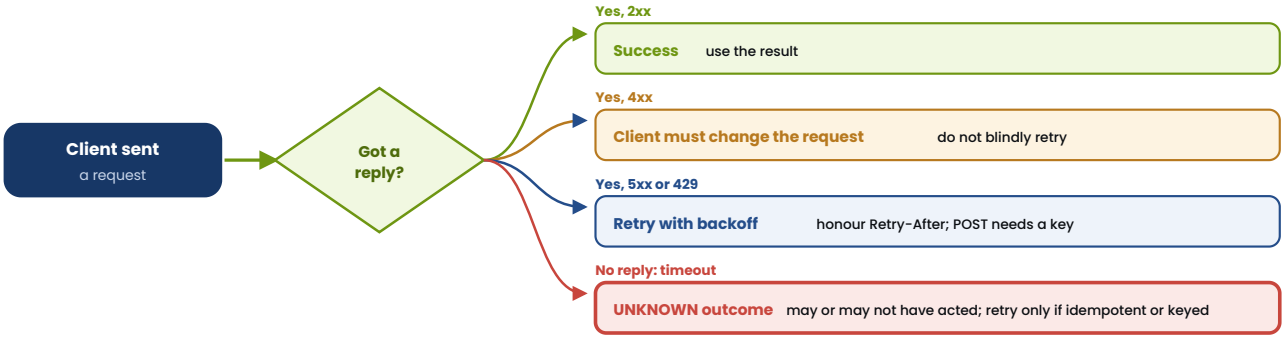
MEMORY HOOK: ONE HUNDRED TO ONE

39 creates and **3,900** redirects a second: the redirect is the hot path, so it is the one to optimise (Lesson 6).

TWO MORE PICTURES TO REMEMBER

Which side of the fence is the failure on?

You have seen the URL journey (Concept 1), the idempotent retry (Concept 6) and the API-style decision tree (Concept 8). Here are two more.



A timeout is the dangerous branch, because the client cannot tell lost request from lost reply.



The anatomy of one API call. Every API decision lives in one of these slots: method, path, query, headers, body, status code.

STORY RECAP: THE TUNNEL, SOLVED

The phone in the tunnel sent `POST /pay` with a key. It timed out: the bottom branch, outcome unknown. Because the request was **keyed**, it retried with the same key. The server found the key, replayed the stored result, and charged **₹500 once**.

MEMORY HOOK: READ THE FENCE

Got a reply? Read the code. No reply? Assume **unknown**, and retry only what is idempotent or keyed.

INTERVIEW MODE: SAY IT LIKE THIS

- "A timeout is not an error code; it is an unknown outcome, so I only retry what is idempotent or keyed."

THINGS THAT SOUND RIGHT AND ARE NOT

Common misconceptions

BELIEF	CORRECTION
"REST means JSON over HTTP."	REST is a *style*: resources as URLs, standard methods, stateless. JSON is just the usual format. Many "REST" APIs are only REST-like [X6].
"POST is for creating, PUT is for updating, end of story."	PUT replaces a resource at a known URL and is idempotent; POST submits data and is not [S1]. A POST can create or trigger an action.
"Idempotent means safe."	DELETE is idempotent but not safe; it changes state [S1]. Safe means read-only.
"200 with an error message in the body is fine."	Clients, caches and monitors read the status code first. A false 200 hides failures. It is the HTTP version of "I am fine" said through gritted teeth.
"Retry is always safe."	Only idempotent requests, or keyed POSTs, are safe to retry blindly [S2, X7].
"Page numbers (?page=2) are fine for any list."	On changing data they duplicate and skip items. Use cursors for feeds and busy lists [S4].
"gRPC is just a faster REST."	It is a different model: typed methods from a .proto contract, protobuf by default [S5]. Faster is a trade-off, not a law.
"401 and 403 are the same."	401 is about identity (not authenticated); 403 is about permission. Mixing them confuses clients.
"The Idempotency-Key header is part of HTTP."	MDN marks it experimental and non-standard, with a draft specification [S2].

TINY CHALLENGE

A teammate says: "DELETE is idempotent, so we can expose it as a GET link." What is wrong? **Answer: Idempotent is not safe. GET must be read-only, because browsers, proxies and retry logic treat it as harmless to repeat; DELETE changes state.**

INTERVIEW MODE: SAY IT LIKE THIS

- "I choose methods by safety and idempotency, return honest status codes, make retries safe with keys, and paginate changing lists with cursors."

TEAR OUT THIS PAGE

One-page cheat sheet

WHAT THEY ASK	THE ANSWER	THE TRAP
"What happens when you type a URL?"	DNS, TCP, TLS, HTTP request and reply, render	Skipping DNS or the handshake cost
"GET vs POST?"	GET reads, is safe and idempotent; POST submits data, may change state, not idempotent	Saying "GET is for small data"
"PUT vs PATCH vs POST?"	PUT replaces the whole resource; PATCH changes part; POST creates or triggers	Calling PATCH idempotent without qualification (MDN marks it not idempotent)
"What is idempotency?"	Repeating gives the same server state as doing it once	Confusing it with "safe"
"Make a payment API retry-safe."	Idempotency key; store key plus result; same key returns stored result	New key on each retry
"401 vs 403?"	401 not authenticated; 403 authenticated but not allowed	Using 404 or 200 for both
"What does 429 mean?"	Too many requests; wait per Retry-After	Retrying instantly in a loop
"Offset vs cursor?"	Offset skips N and breaks when data changes; cursor continues after an item	Cursor on a non-unique column
"REST vs gRPC vs GraphQL?"	REST for public simple APIs, gRPC for internal typed calls, GraphQL for flexible field selection	"X is always better"
"Why stateless?"	Any server can handle any request, so scaling out is easy	Storing user state in one server's memory
"Authentication vs authorization?"	Who you are vs what you may do	Mixing the order
"How do you version an API?"	A /v1/ prefix is common; evolve without breaking old clients	Changing a response shape in place

MEMORY HOOK: THE JOURNEY

D-T-T-H: Do Take The Highway (DNS, TCP, TLS, HTTP).

MEMORY HOOK: THE METHODS

PUT = **"set to"**, POST = **"add"**. Only "set to" is safe to nag twice.

MEMORY HOOK: THE CODES

2 yes, 3 go there, 4 you, 5 us. **401 Who? 403 No.**

WHAT YOU WILL ACTUALLY TYPE AT WORK

Where this shows up in real systems

curl -i shows the real thing

curl -i https://example.com prints the status line and response headers before the body. curl -v also shows the TLS handshake and the request headers sent.

curl -w splits latency

Real curl write-out variables split a request into the Concept 1 steps (below).

Browser DevTools, Network tab

Each request shows its method, status code, and a timing breakdown (DNS, connection, TLS, waiting for the server, download). Label names vary by browser version.

Stripe's idempotent POST

All POST requests accept keys, but GET and DELETE should not carry them, since they are idempotent by definition [S3].

Stripe's cursor pagination

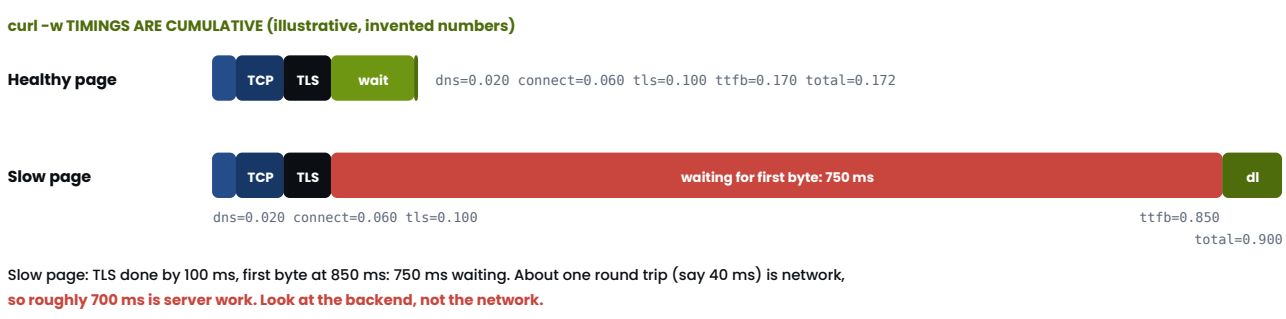
limit plus starting_after; the reply has "has_more": false on the last page [S4].

A rate-limited reply

HTTP/1.1 429 Too Many Requests plus Retry-After: 60, as in MDN's 429 example (which shows Retry-After: 3600) [X2].

```
curl -i https://example.com                # status line + headers, then body
curl -s -o /dev/null -w "dns=%{time_namelookup} connect=%{time_connect} \
  tls=%{time_appconnect} ttfb=%{time_starttransfer} total=%{time_total}\n" https://example.com
# illustrative output (invented): dns=0.020 connect=0.060 tls=0.100 ttfb=0.170 total=0.172
curl https://api.stripe.com/v1/customers -H "Idempotency-Key: KG5LxwFBepaKHyUD" ...   # [S3]
curl -G https://api.stripe.com/v1/customers -d limit=3 -d starting_after=<id>   # [S4]
```

Read the curl -w line as: DNS 20 ms, TCP done at 60 ms, TLS done at 100 ms, first byte at 170 ms. The values are cumulative seconds. Verify the variable meanings in curl's manual before relying on subtractions.



Reading curl -w. Top: the healthy output above. Bottom: the slow page from Scenario 8.

INTERVIEW MODE: SAY IT LIKE THIS

- "I inspect an API call with curl: status line, headers, timing. Real APIs such as Stripe use idempotency keys for POST and cursor pagination for lists."

FUNDAMENTALS WITH MODEL ANSWERS

Interview questions 1 and 2

Very few candidates quote verbatim "REST" questions; most mention them as follow-ups inside a design round, so several here are Pattern-only, and the label says so.

Q1 REST vs gRPC vs GraphQL SOURCE-CONFIRMED

Cisco, onsite whiteboard, Senior role, interview Jul 2026 (reported as follow-up topics while the candidate drew a system). glassdoor.com.br/Entrevista/Cisco-Entrevista-E1425-RVW103576482.htm [S6]

Model answer. "REST names resources with URLs and uses HTTP methods; it is simple, widely understood, and good for public APIs. gRPC defines methods in a proto file and generates client and server code, using protobuf by default, so it suits internal service-to-service calls. GraphQL lets the client pick the fields it needs from typically one endpoint, which helps when many screens need different data. I would pick by who calls the API and what shape the data has."

Why gRPC inside the company? "Both sides share a typed contract in the proto file, and generated code removes hand-written parsing; messages are compact [S5]. Callers are our own services, so we control both ends."

A screen needs a user and their last 3 orders. Compare REST and GraphQL. "With 40 ms RTT and 30 ms server time per call: two parallel REST calls take about 70 ms; two dependent sequential REST calls about 140 ms; one GraphQL query about 70–80 ms. GraphQL helps mostly when calls would be dependent."

200 internal services plus a public API? "gRPC between services, REST at the public edge. The cost is operating two styles, plus a gateway to translate. I would add rate limits and versioning at the edge."

Weak: "gRPC is newest so it is best." (Newest is a release date, not an architecture.)

Strong: "Public simple resources: REST. Internal typed calls: gRPC. Flexible clients: GraphQL with query-cost limits."

Q2 How would you paginate an API? CANDIDATE-REPORTED

Uber, SDE-intern, post dated Jul 2025; the post's style looks templated. jointaro.com/interviews/companies/uber/experiences/sde-intern-bangalore-rural-july-1-2025-accepted-offer-positive-5441773a/ [X1]

Model answer. "I would use cursor pagination: limit plus an after cursor that is the last item's ID, and a has_more flag in the reply. Offset pagination breaks when data changes between pages and slows down for deep pages."

Why does offset break? "The offset counts positions, and inserts or deletes shift positions. Three new posts at the top make page 2 repeat three items."

Page size 10, three posts inserted. What does the user see? "Page 2 starts at position 11, which is now the old position 8. Three duplicates, so 7 new items. Cursor starting_after=91 gives 90 downward, no duplicates."

Two posts share the same timestamp. Problem? "A cursor on a non-unique column can skip or repeat items. Sort by (created_at, id) and make the cursor carry both."

Weak: "Use page numbers." (Bold choice on a feed that updates every second.)

Strong: "Cursor on a unique sort key, limit, has_more, and I can say what offset breaks."

MEMORY HOOK: THE ANSWER SHAPE

Every strong answer here has the same four parts: **a one-line definition**, **a number**, **a trade-off**, and **what you would choose**.

FUNDAMENTALS WITH MODEL ANSWERS

Interview questions 3 to 5

Q3 What happens when you type a URL into the browser? CANDIDATE-REPORTED

Amazon: a candidate says they were asked it (LinkedIn post, about 2023) [X3]. Flipkart's SDE-1 hiring-manager round also covers computer networks generally (loop in 2024, posted Mar 2026) [S10].

Model answer. "The browser asks DNS for the IP address, opens a TCP connection, does a TLS handshake for HTTPS, sends the HTTP request, the server processes it and replies with a status code and body, and the browser renders it."

> **Why does the handshake matter?** "Each round trip before data adds latency. With 40 ms RTT, TCP plus TLS 1.3 add 80 ms before the request even leaves."

> **First-request and second-request times?** "With DNS 20 ms, RTT 40 ms, server 30 ms: first is 20 + 40 + 40 + 40 + 30 = 170 ms; on a reused connection it is 40 + 30 = 70 ms."

> **How would you make it faster?** "Reuse connections, cache DNS, put static content near users, and cut round trips. The cost is complexity and stale caches, which later lessons cover."

Weak: "It loads the website." (Technically true, like saying cooking is "food appears".)

Strong: "DNS, TCP, TLS, HTTP, render, and I can price each step."

Q4 PUT vs POST (and PATCH) PATTERN-ONLY

Company: none found. A staple of REST question lists with no company attached [X6].

Model answer. "PUT replaces the resource at a known URL and is idempotent. POST submits data to a resource, often creating something, and is not idempotent. PATCH changes part of a resource and is not idempotent [S1]."

> **Why is PUT idempotent?** "Sending the same full replacement twice leaves the same state: 'set balance to 150' twice is still 150."

> **Balance 100. Send "credit 50" by POST twice.** "200. The same intent as PUT-to-150 sent twice would still be 150."

> **How do you make that POST retry-safe?** "Add an idempotency key and store key plus result. A repeat returns the stored result [S2]."

Weak: "POST creates, PUT updates."

Strong: "PUT replaces and repeats safely; POST does not, so it needs a key."

Q5 Make a payment API safe to retry PATTERN-ONLY

None found as a verbatim question, so the question is Pattern-only. Context: an Amazon SDE-2 preparation list includes "High-Level Design of a Payment Service (focus on scalability, fault tolerance, trade-offs)" [S9], a design prompt, not this question; that list item is Candidate-reported. leetcode.com/discuss/post/6906753/amazon-hld-lld-dsa-questions-by-anonymou-sepk/

Model answer. "The client generates a unique idempotency key and sends it with the payment POST. The server checks a key store: if the key is new, it charges and stores the result under the key; if it exists, it returns the stored result without charging again. The client reuses the same key on every retry."

> **Why not just retry?** "A lost reply looks identical to a lost request. Without a key, a retry after a lost reply charges twice."

> **10 million payments a day, 36-byte keys, 1 KB replies, kept 24 hours. Storage?** "10,000,000 x (36 + 1,000) bytes = 10,360,000,000 bytes, about 10.4 GB. Average rate: 10,000,000 / 86,400 = about 116 requests per second."

> **Two requests with the same key arrive together.** "The key store must be atomic: the first inserts a 'processing' marker; the second sees it and gets 409 (MDN's suggested code) and retries later [S2]."

Weak: "Check if the payment already happened."

Strong: "Key, atomic store, stored result, same key on retry, conflict code for in-flight duplicates."

FUNDAMENTALS WITH MODEL ANSWERS

Interview questions 6 to 8

Q6 401 vs 403, and 404 vs 410 PATTERN-ONLY

Company: none found.

Model answer. "401 means the request lacks valid credentials: not authenticated. 403 means the caller is known but not allowed. 404 means no such resource; 410 Gone means it existed and was removed on purpose."

> **Why distinguish them?** "The client's next action differs: for 401 it logs in again; for 403 logging in again will not help."

> **Ever return 404 for a resource the user may not see?** "Sometimes deliberately, to avoid revealing existence. The cost is harder debugging. I would state the choice."

> **An expired short link: which code?** "410 if I know it expired; 404 if I treat unknown and expired alike."

Weak: "Both mean access denied."

Strong: "401 identity, 403 permission, and the client action differs."

Q7 "Walk me through the API for this system" CANDIDATE-REPORTED

Atlassian, SDE-2, India, page undated: cover requirements, then API design including protocols, then database design [S7]. A Cisco campus guide (an aggregator) also lists API design among expected topics for software roles [X8]. leetcode.com/discuss/interview-experience/1795183/atlassian-sde-2-remote-india · faceprep.in/article/cisco-hiring-process-for-freshers-2026/

Model answer. "I list the entities, then four or five endpoints with method, URL, request body, response and error codes. I add cursor pagination to lists, an idempotency key to important POSTs, authentication in the Authorization header, and a 429 with Retry-After for limits. I state the protocol: REST over HTTPS for the public API."

> **Why only four or five endpoints?** "A design round has limited time; the interviewer can ask for more [S11]."

> **Estimate the traffic for your list endpoint.** "I tie it to the earlier scale numbers; for the short-link service about 3,900 redirects per second dwarfs the list endpoint."

> **Would you version the API?** "Yes: a /v1/ prefix lets me change shapes later without breaking old clients; the cost is maintaining old versions."

Weak: "I would make CRUD endpoints."

Strong: "Endpoints with errors, pagination, idempotency, auth and limits, each justified."

Q8 Authentication vs authorization PATTERN-ONLY

Company: none found. Appears in REST and API-testing question lists [X6].

Model answer. "Authentication proves who you are, usually with a token in the Authorization header. Authorization decides what you may do. You can be authenticated yet forbidden, which is the 403 case."

> **Why keep them separate?** "Different failures need different fixes: bad credentials versus insufficient permission."

> **A user's token expires mid-session.** "The API returns 401; the client refreshes or re-authenticates, then retries the request. For a keyed POST, it reuses the same key."

> **Session vs token across many servers?** "Sessions need shared storage so any server can look them up; signed tokens avoid the lookup but are harder to revoke early. I would state the trade-off I accept." (Standard background; no source opened for this comparison.)

Weak: "Login and permissions."

Strong: "Who versus what, 401 versus 403, and I can name a trade-off for sessions versus tokens."

HOW TO THINK WHEN IT BREAKS

Scenario questions 1 to 4

Each shows *how to think*. Labels refer to the underlying situation; the step lists are original.

1. Charged twice after tapping Pay once. (The most expensive double-tap in tech.) PATTERN-ONLY

1. **Reconstruct the timeline** from logs: how many POST /pay requests arrived for that order, and with which idempotency keys?
2. **Same key twice:** the server should have returned the stored result. If it charged twice, the key check is not atomic, or keys expired.
3. **Different keys:** the client generated a new key per retry. Fix the client to reuse one key per user action.
4. **No keys at all:** add keys to the endpoint (Concept 6), and make the server reject payment POSTs without one (400, per MDN's suggestion [S2]).
5. **Repair:** refund the duplicate, then add an alert that counts duplicate charges per order.

Why this works: you separated the three causes (no key, new key, broken key store) instead of guessing.

2. Dashboards show 100% success, but users report errors. PATTERN-ONLY

1. Check whether the API returns 200 with {"error": ...} in the body. Dashboards usually count status codes.
2. Change the API to return 4xx for client mistakes and 5xx for server faults.
3. Update clients to read the status code first.
4. Add alerts on the 5xx rate and on the 4xx rate per endpoint.

3. A mobile feed shows duplicate posts and sometimes skips posts. PATTERN-ONLY

1. Ask how the API pages: ?page=2 or offset means positions shift when posts arrive (Concept 7).
2. Show the numbers: page size 10, 3 new posts, page 2 repeats 3 items.
3. Switch to cursor pagination on a unique sort key (created_at, id), with limit and has_more [S4].
4. Keep the old offset parameters for a deprecation window, using a new /v2/ or a new parameter, so old clients do not break.

4. After a deployment, intermittent 502 and 504 errors. PATTERN-ONLY

1. Decode: 502 Bad Gateway means a proxy or gateway got a bad reply from an upstream; 504 Gateway Timeout means the upstream was too slow (names from MDN's list [S1 nav list]; meanings are standard usage).
2. Compare error timing with the deployment time and with which service instances were replaced.
3. Check the gateway's timeout against the upstream's slowest normal time (p99). A gateway timeout shorter than the upstream's p99 creates 504s.
4. If new instances take time to warm up, check health checks are not sending traffic too early.
5. If it started exactly at the deployment, roll back first, investigate after.

HOW TO THINK WHEN IT BREAKS

Scenario questions 5 to 8

5. A third-party API keeps returning 429. (Retrying instantly is ringing a doorbell 50 times.) PATTERN-ONLY

1. Read Retry-After if present and wait that long [X2].
2. Without it, use exponential backoff with jitter, never retrying instantly or in lockstep with other clients [X7].
3. Cut your request rate and batch where the API allows (one request for ten items beats ten requests) [X7].
4. For POST, retry only with an idempotency key, since a 429 can arrive after the work was done [X7].

6. "Design the API for a URL shortener." SOURCE-CONFIRMED

1. Clarify scope: create, redirect, list, delete in scope; analytics and custom aliases out (Lesson 1).
2. Write five endpoints (the running example): create with an idempotency key, redirect, list with a cursor, get, delete.
3. Choose status codes: 201 with Location, 302 redirect, 404/410, 204, 429 with Retry-After.
4. Add authentication in the Authorization header and a /v1/ prefix.
5. State the numbers: about 39 creates and 3,900 redirects per second on average.

Reported as: Amazon (Embedded Software Dev Engineer role), interview Jun 2025, system design as the last of four rounds [S8]. glassdoor.co.in/Interview/Amazon-Interview-E6036-RVW102556955.htm

7. A logged-in user gets 403 on their own link. PATTERN-ONLY

1. Confirm the code: 403 means known but not allowed, so authentication works.
2. Check ownership: does the link's owner ID equal the caller's ID?
3. Check token details: scopes or roles in the token versus what the endpoint requires.
4. Check the authorization rule itself for off-by-one logic, such as comparing a string ID to a number.

8. A page feels slow. Where is the time going? PATTERN-ONLY

1. Run the curl -w command from the Real systems page several times.
2. Illustrative output: dns=0.020 connect=0.060 tls=0.100 ttfb=0.850 total=0.900.
3. Read it: DNS 20 ms, connection by 60 ms, TLS by 100 ms (all small). First byte at 850 ms, so waiting after TLS is 850 - 100 = 750 ms.
4. About one round trip of that is network (say 40 ms), so the rest, roughly 700 ms, is server work. Look at the backend, not the network.

Why this works: you used measurements to choose where to look.

THE PATTERN BEHIND EVERY SCENARIO

Decode the status code or timing, **separate** the possible causes, **check** the one the evidence points to, then **fix** and add an alert so it cannot hide again.

SOLVE THESE BEFORE YOU LOOK

Practice problems 1 to 3

Original problems, not reported questions. Cover the solution, try it on paper, then check.

P1 Safe and idempotent? EASY

Fill in "safe?" and "idempotent?" for GET, HEAD, PUT, DELETE, POST, PATCH.

> **Hint.** Safe means read-only. Idempotent means repeating leaves the same server state.

SOLUTION

GET: safe, idempotent. HEAD: safe, idempotent. PUT: not safe, idempotent. DELETE: not safe, idempotent. POST: not safe, not idempotent. PATCH: not safe, not idempotent. (MDN's table [S1].)

P2 Pick the status code EASY

(a) a new link is created; (b) a link is deleted and there is nothing to return; (c) the body is invalid JSON; (d) no token was sent; (e) valid token, but the link belongs to someone else; (f) unknown short code; (g) the 61st request in a minute against a limit of 60; (h) the server hit an unexpected bug; (i) the gateway's upstream took too long.

> **Hint.** Classes: 2xx success, 4xx the client's fault, 5xx the server's.

SOLUTION

(a) 201. (b) 204. (c) 400. (d) 401. (e) 403. (f) 404. (g) 429 with Retry-After. (h) 500. (i) 504.

P3 Connection cost NUMERICAL

RTT = 100 ms, DNS = 30 ms, server work = 50 ms. Compute the first-request time with TLS 1.2 (two round trips) and TLS 1.3 (one round trip), and the time for a second request on the same connection. What percentage does reuse save under TLS 1.3?

> **Hint.** First request = DNS + TCP (1 RTT) + TLS + request (1 RTT) + server work. Reused connection = request (1 RTT) + server work.

SOLUTION

TLS 1.2: 30 + 100 + 200 + 100 + 50 = **480 ms**. TLS 1.3: 30 + 100 + 100 + 100 + 50 = **380 ms**. Reused: 100 + 50 = **150 ms**. Savings under TLS 1.3: 380 - 150 = 230 ms, and 230 / 380 = about **60.5%**. (Typical handshake costs; verify the TLS version your servers actually use.)

HOW TO CHECK YOUR OWN ANSWER

For a status code, say which **family** it is in and what the client does next. For a time, write **one line per step** with its unit. For a list, ask what happens when items **arrive or vanish** between pages.

SOLVE THESE BEFORE YOU LOOK

Practice problems 4 and 5

P4 Offset duplicates NUMERICAL

A feed has 1,000 items, newest first, page size 25. You have read pages 1–3 (positions 1–75). Then 7 new items arrive at the top. (a) How many pages in total? (b) With offset paging, what does page 4 (offset 75) return? (c) With cursor paging, what does the next page return?

> **Hint.** After 7 inserts, new position p holds what used to be at p - 7.

SOLUTION

(a) 1,000 / 25 = **40 pages** (before the inserts). (b) Offset 75 returns new positions 76–100, which are old positions 69–93. Old positions 69–75 were already on page 3, so **7 duplicates** and 18 genuinely new items (old 76–93). (c) The cursor is the last item of page 3 (old position 75), so the next page is old positions 76–100: **25 new items, no duplicates**.

P5 Design a notes API MEDIUM

A "notes" app lets users create notes, list their notes, edit a note, delete a note, and share a note read-only with another user. Write the endpoints with methods, URLs, success codes and error codes, plus pagination, idempotency and rate-limit handling.

> **Hint.** One endpoint per feature. Sharing creates a new thing, so what resource is it?

SOLUTION

One valid answer:

ENDPOINT	SUCCESS	ERRORS / NOTES
`POST /v1/notes` + Idempotency-Key	201	400, 401, 422, 429
`GET /v1/notes?limit=50&starting_after=<id>`	200 with data[], has_more	
`GET /v1/notes/{id}`	200	403 if not owner or shared, 404
`PUT /v1/notes/{id}` (replace) or `PATCH` (partial)	200	409 if edit conflict
`DELETE /v1/notes/{id}`	204	
`POST /v1/notes/{id}/shares` {"user_id": "…", "access": "read"}	201	sharing is a resource
`DELETE /v1/notes/{id}/shares/{user_id}`	204	

All endpoints: authentication in Authorization, 429 with Retry-After when limited. The share resource is a noun, not a verb like /shareNote.

SOLVE THIS BEFORE YOU LOOK

Practice problem 6

P6 Size and design an idempotency store HARD

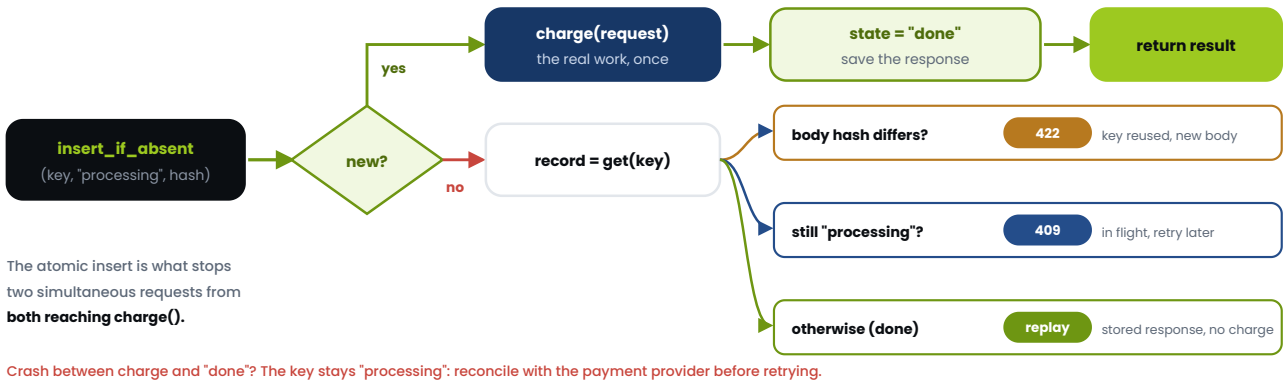
A payments API receives 10 million POST /pay requests a day. Keys are 36-byte UUID strings; the stored reply is 1 KB; keys live 24 hours. (a) Average requests per second? (b) Peak at 5x average? (c) Storage for 24 hours? (d) For 7 days? (e) Write the server logic so two simultaneous requests with the same key do not both charge.

Hint. Seconds per day = 86,400. For (e), you need an *atomic* "insert if absent".

SOLUTION

(a) 10,000,000 / 86,400 = 115.7, so about **116 per second**. (b) 5 x 116 = about **580 per second**. (c) 10,000,000 x (36 + 1,000) = 10,360,000,000 bytes = about **10.4 GB**. (d) 7 x 10.36 GB = about **72.5 GB**. (e) Pseudo-code, language-neutral, below.

```
handle(request, key):
    ok = store.insert_if_absent(key, state="processing", fingerprint=hash(request.body))
    if not ok:
        record = store.get(key)
        if record.fingerprint != hash(request.body): return 422
        if record.state == "processing":             return 409   # in flight, retry later
        return record.saved_response                 # done: replay stored result
    result = charge(request)                         # do the real work once
    store.update(key, state="done", saved_response=result)
    return result
```



The same logic as a flowchart. The 409 and 422 behaviours follow MDN's description [S2].

The atomic insert_if_absent is what stops two simultaneous requests from both reaching charge. If the server crashes between charge and update, the key stays "processing": design a recovery step (for example, check the payment provider for the charge before retrying), which is why real systems also reconcile.

TEN QUESTIONS, TWO MINUTES EACH

Mini quiz

Original questions. Cover the answers, answer all ten, then check.

1. Which methods are idempotent (per MDN)?
A) GET, PUT, DELETE B) GET, POST, PUT C) PUT, POST, PATCH D) GET, POST, DELETE
Answer A. GET, PUT and DELETE are idempotent [S1]. B, C and D each include POST, and C includes PATCH; both are not idempotent.

2. A client sends a POST and times out with no reply. What is true?
A) The server did nothing B) The server did the work C) The outcome is unknown D) The request was invalid
Answer C. A timeout cannot distinguish a lost request from a lost reply. A and B each assume one case; D has no basis, since an invalid request would get a 4xx reply.

3. Valid JSON that fails the business validation rules is best answered with:
A) 200 B) 404 C) 422 D) 500
Answer C. 422 Unprocessable Content fits "well-formed but invalid" (400 for malformed input is also common). A hides the failure; B says the resource is missing; D blames the server.

4. Offset pagination, page size 10, 3 new items at the top between page 1 and page 2. Duplicates on page 2?
A) 0 B) 3 C) 7 D) 10
Answer B. Page 2 starts at position 11, which now holds old position 8; old positions 8–10 repeat. A ignores the shift; C and D mix up new items with duplicates.

5. What does MDN say about the Idempotency-Key header?
A) A core HTTP standard B) Experimental and non-standard, with a draft spec C) Only for GET D) Obsolete
Answer B [S2]. A overstates its status; C is wrong because it exists for POST and PATCH; D is wrong because it is current, just not standardised.

6. RTT 40 ms, DNS 20 ms, server 30 ms, TLS 1.3: how much does reusing the connection save?
A) 40 ms B) 70 ms C) 100 ms D) 170 ms
Answer C. First: 20 + 40 + 40 + 40 + 30 = 170 ms. Reused: 40 + 30 = 70 ms. Saved: 100 ms. B is the reused time; D the first-request time; A one handshake only.

7. gRPC uses which data format by default?
A) XML B) Plain text C) Protocol Buffers D) YAML
Answer C [S5]. Protobuf by default, though other formats such as JSON are possible. A, B and D are not its default.

8. Asha is logged in and tries to delete Ravi's link. Best response?
A) 401 B) 403 C) 404 D) 200
Answer B. Authenticated but not authorized. A implies missing credentials; C hides existence (sometimes deliberate, not the standard answer); D is a false success.

9. A retry of a payment POST should use:
A) A new key B) No key C) The same idempotency key as the first attempt D) A random method
Answer C. The same key lets the server recognise the repeat [S2, S3]. A makes the retry look new and can double-charge; B removes protection; D is meaningless.

10. A short-link service wants to count every click. Better default redirect?
A) 301 B) 302 C) 200 D) 204
Answer B. A temporary redirect usually keeps traffic flowing through the service; a 301 can be cached by browsers, so later clicks may skip it (typical behaviour; verify for your targets). C and D are not redirects.

FIVE SENTENCES TO KEEP

Recap and the 60-second answer

1. A request crosses DNS, TCP, TLS and HTTP; each step costs time, and a timeout leaves the outcome unknown.
2. HTTP is stateless; resources get nouns in URLs, and methods carry meaning: GET, PUT and DELETE are idempotent, POST and PATCH are not.
3. Status codes drive client behaviour: 2xx success, 4xx the client's fault (401 identity, 403 permission, 429 too many), 5xx the server's.
4. Make retries safe with idempotency keys; paginate changing lists with cursors on a unique sort key.
5. Pick REST for simple public APIs, gRPC for internal typed calls, GraphQL for flexible field selection; always state the trade-off.

SAY THIS IN AN INTERVIEW: "HOW WOULD YOU DESIGN THE API FOR THIS SERVICE?"

"I list the entities, then four or five endpoints. I use nouns in URLs and the right methods: GET to read, POST to create, PUT or PATCH to change, DELETE to remove. I return honest status codes, such as 201 with a Location header on create, 404 for unknown resources, 401 for missing credentials, 403 for forbidden, and 429 with Retry-After when rate-limited. Every list uses cursor pagination with a limit and a has-more flag, because offset breaks when data changes. Any POST that moves money or creates something important takes an idempotency key, so a retry after a lost reply cannot do the work twice. The API is stateless, uses HTTPS, carries credentials in the Authorization header, and has a version prefix. For internal service calls I might choose gRPC, for flexible clients GraphQL; I would state the cost of whichever I pick."

MEMORY HOOK: TUNNEL

Lost request or lost reply? **The client can't know**, so key every important POST.

MEMORY HOOK: WAITER

The waiter forgets: **every slip carries everything** (stateless).

MEMORY HOOK: FEED

Bookmarks, not seat numbers (cursor, not offset).

COVERAGE MAP

SUB-TOPIC	COVERED HERE	WHERE ELSE
Request/response journey, DNS, TCP, TLS	Overview with numbers	Lesson 5 (DNS, load balancers)
HTTP messages, methods, status codes	Fully	None needed
Idempotency and keys	Fully	Lesson 10 (queues and consumers), Lesson 11 (retries)
Pagination	Fully	None needed
REST vs gRPC vs GraphQL	Compare and choose	Lesson 31 (internal services)
Authentication, authorization, HTTPS	10-minute version	Not a full lesson
API recipe	Fully	Lessons 24 to 32 (per-system APIs)

CHECK OUR WORK

Sources

S = opened in full with a fetch tool (dates where the page gives one). X = seen only in search-result extracts. Every latency, pagination and storage figure in this lesson was recomputed.

OPENED IN FULL

ID	PAGE	WHAT IT SUPPORTED	URL
S1	MDN: HTTP request methods (last modified 23 Sep 2026)	Safe/idempotent table, method descriptions, header and status-code name lists	developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Methods
S2	MDN: Idempotency-Key header (last modified 4 Jul 2026)	Mechanism, server error codes 400/409/422, "experimental, non-standard"	developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Idempotency-Key
S3	Stripe docs: Idempotent requests	Stored first result, 255-character keys, pruning after at least 24 hours, POST keys, GET/DELETE need none	docs.stripe.com/idempotency
S4	Stripe docs: Pagination	limit, starting_after, ending_before, has_more, cursor pagination	docs.stripe.com/api/pagination?cat=-1
S5	gRPC: Introduction (modified 12 Nov 2024)	RPC model, proto files, protobuf by default	grpc.io/docs/what-is-grpc/introduction/
S6	Glassdoor: Cisco Senior SWE reports (incl. Jul 2026)	REST vs gRPC vs GraphQL, QPS, rate limiting, circuit breaker follow-ups; Apr 2026 networking screening	glassdoor.com.br/Entrevista/Cisco-Entrevista-E1425-RVW103576482.htm
S7	LeetCode: Atlassian SDE-2, India (page undated)	Design round order: requirements, API with protocols, database, services	leetcode.com/discuss/interview-experience/1795183/atlassian-sde-2-remote-india
S8	Glassdoor: Amazon Embedded SDE (interview Jun 2025)	URL shortener as the last of four rounds	glassdoor.co.in/Interview/Amazon-Interview-E6036-RVW102556955.htm
S9	LeetCode: "Amazon HLD LLD DSA Questions" (SDE-2 list)	Payment-service HLD prompt in a list	leetcode.com/discuss/post/6906753/amazon-hld-lld-dsa-questions-by-anonymou-sepk/
S10	LeetCode: Flipkart SDE-1 (posted Mar 2026; loop in 2024)	HM round covers computer networks, OOPs, DBMS, OS	leetcode.com/discuss/post/7691468/flipkart-interview-experience-for-sde-1-bgs2k/
S11	DesignGurus: Four-Step Framework (last updated May 2026)	Four or five endpoints, not fifteen; time budgets	designgurus.io/system-design-interview/framework

CHECK OUR WORK

Sources, gaps and a self-check

SEEN ONLY IN SEARCH-RESULT EXTRACTS (PAGES NOT SEPARATELY OPENED)

ID	WHAT IT SUPPORTED	URL
X1	Uber pagination question (Taro, Jul 2025; templated-looking post)	jointaro.com/interviews/companies/uber/experiences/sde-intern-bangalore-rural-july-1-2025-accepted-offer-positive-5441773a/
X2	429 and Retry-After: MDN German page (429 definition and example), Portuguese page (cites RFC 6585 section 4), Japanese page (Retry-After uses)	developer.mozilla.org/de/docs/Web/HTTP/Status/429 · developer.mozilla.org/pt-BR/docs/Web/HTTP/Status/429 · developer.mozilla.org/ja/docs/Web/HTTP/Headers/Retry-After
X3	Amazon "What happens when you type a URL" (LinkedIn post, about 2023)	linkedin.com/posts/kprasadrao_a-common-interview-question-that-even-i-got-activity-7064947742992674819-t41k
X4	gRPC over HTTP/2, multiplexing and streaming (secondary explainers)	integralist.co.uk/posts/grpc-for-beginners/index.md · mattermost.com/blog/an-introduction-to-grpc/
X5	GraphQL single endpoint, client-chosen fields, query-cost limits (secondary explainers)	walturn.com/insights/best-practices-for-graphql-apis · hayksimonyan.substack.com/p/graphql-api-basics-and-best-practices
X6	REST interview question lists; REST-like vs fully RESTful (DataCamp, updated Apr 2026)	datacamp.com/it/blog/rest-api-interview-questions
X7	Retrying after 429: backoff, jitter, keys for POST, batching (ApyHub)	apyhub.com/blog/what-does-429-too-many-requests-mean
X8	Cisco campus hiring guide for freshers 2026 (aggregator)	faceprep.in/article/cisco-hiring-process-for-freshers-2026/

Honest gaps: things interviewers sometimes reach that this lesson only touches or leaves out

- TCP vs UDP internals and IP subnetting: Cisco's Bengaluru screening in Apr 2026 asked about them [S6]; that belongs to Computer Networks
- HTTP/2 and HTTP/3 details, WebSockets, server-sent events (Lesson 26 touches real-time delivery)
- OAuth flows, JWT internals, cookies, CORS: only named, not taught
- Caching headers (Cache-Control, ETag): Lesson 6. API gateways and load balancers: Lesson 5
- Long-running operations (202 Accepted), webhooks, file uploads, versioning strategies, OpenAPI, GraphQL subscriptions, gRPC streaming types
- Sessions vs tokens is standard background with no source opened. Not claimed: that nothing else can be asked

SELF-CHECK: CAN YOU SAY YES TO ALL FOUR?

I can price a request
DNS, TCP, TLS, HTTP, with milliseconds.

I can pick methods and codes
Safe vs idempotent, 401 vs 403, 429 + Retry-After.

I can make a POST retry-safe
Same key, atomic store, stored result.

I can sketch an API in 5 minutes
Cursor lists, keyed POSTs, limits, auth, /v1/.

TEAR OFF AND KEEP

Your lesson 2 memory card

D-T-T-H
Do Take The Highway: DNS, TCP, TLS, HTTP. Only the last one carries your data.

LOST REQUEST OR LOST REPLY?
The client can't know. Key every important POST.

SET TO VS ADD
PUT "set to 150" repeats safely; POST "add 50" does not.

2 · 3 · 4 · 5
Yes · go there · you messed up · we messed up.

401 WHO? 403 NO.
Identity vs permission; the client's next move differs.

BANK TOKEN
Same idempotency key, same file, no second charge.

BOOKMARKS, NOT SEATS
A cursor names an item; an offset counts positions.

STRANGERS, FAMILY, PICKY EATERS
REST, gRPC, GraphQL.

EVERY ENGINEER LOVES A VERSION
Entities, endpoints, lists-keys-limits, auth, version.

THE FORGETFUL WAITER
Stateless: every request carries everything it needs.

> The train comes out of the tunnel. **Your app retried with the same key, and you paid ₹500 exactly once.**

YOUR NEXT STEPS

STEP 01
Solve problems 1 to 6 on paper

STEP 02
Run curl -i and curl -w on a site you use

STEP 03
Read ahead: Lesson 3, relational databases